Retrospective Sprint IV

Team 04

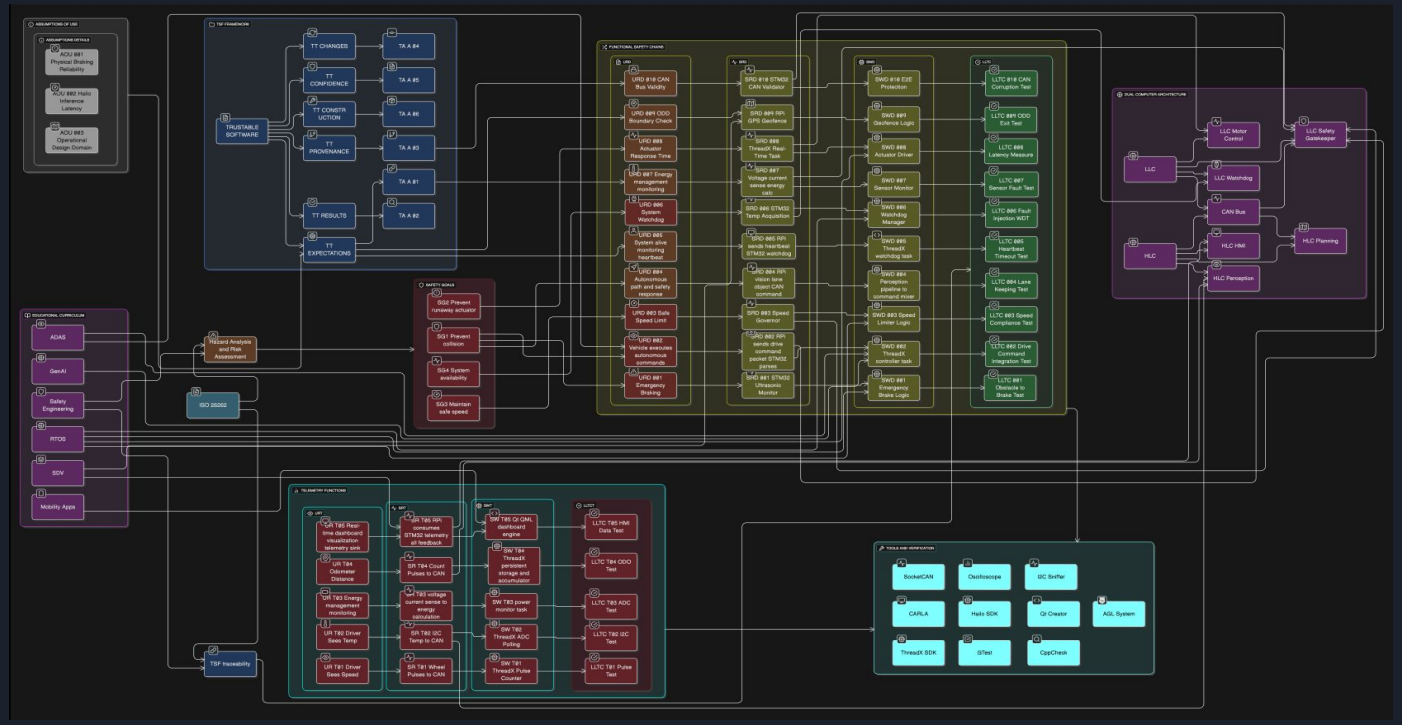
Sprint IV Goal

Finalize the integration of the **RTOS** on **STM32** and **AGL** operating system setup, establish the critical **CAN** bus technical documentation, and conduct essential research (Spikes) into CI automation, system performance, and next-generation automotive architecture (**KUKSA/uProtocol**).

Planning Poker & Fibonacci Estimation

High Engagement: The team actively participated in the poker-playing demo.
Clear Concepts: The connection between the Fibonacci sequence and estimating uncertainty was well-explained.
Great Initiative: Big thanks to Bernardo for preparing and leading the session!

TSF Architecture



Eclipse uProtocol Investigation

Implementing Eclipse **uProtocol** on an **STM32** with Classic **CAN** is not recommended. The CAN bus's 8-byte message limit causes excessive fragmentation and latency for **uProtocol's** larger headers, making it unsuitable for real-time control. A Raspberry Pi 5 should instead act as the **uProtocol** gateway.

GitHub Action STM32

Established robust, event-driven automation for both code verification (CI) and final delivery (CD).

Next Step: Integrate code coverage reporting into the existing CI pipeline to increase code quality metrics.

ThreadX

We successfully established a solid foundation with robust DC motor control logic, and the ThreadX task scheduling is performing as expected. Furthermore, all three components: DC motor, servo, and speed sensor were successfully integrated into a single application.

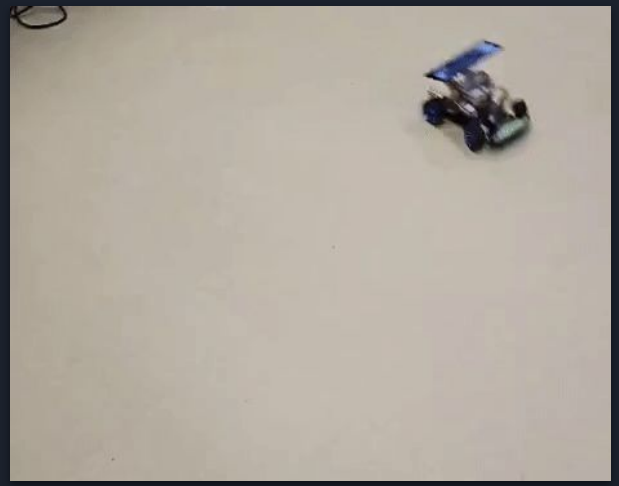
```c
#include "app_threadx.h"

void thread_init(void)
{
    uint8_t status;
    status = 0;
    char err_msg[20] = "FailSP\r\n";

                                        //SUPERVISOR THREAD
    if (tx_thread_create(&threads[supervisor_e].thread_ptr, "led_thread1", ld1_thread_entry, 0, threads[supervisor_e].thread_Stack, THREAD_STACK_SIZE,
    10, 10, TX_NO_TIME_SLICE, TX_AUTO_START) != TX_SUCCESS)
        status = TX_THREAD_ERROR;
    if (status == TX_THREAD_ERROR)
        HAL_UART_Transmit(&huart1, (uint8_t*)err_msg, strlen(err_msg), HAL_MAX_DELAY);

                                        //DC MOTOR THREAD
    if (tx_thread_create(&threads[dc_motor_e].thread_ptr, "motor_thread", dc_motor, 0, threads[dc_motor_e].thread_Stack, THREAD_STACK_SIZE,
    4, 4, TX_NO_TIME_SLICE, TX_AUTO_START) != TX_SUCCESS)
        status = TX_THREAD_ERROR;
    if (status == TX_THREAD_ERROR)
    {
        sprintf(err_msg, "FailDCmt\r\n");
        HAL_UART_Transmit(&huart1, (uint8_t*)err_msg, strlen(err_msg), HAL_MAX_DELAY);
    }

                                        //SERVO MOTOR THREAD
    if (tx_thread_create(&threads[servo_motor_e].thread_ptr, "servo_thread", servo_motor, 0, threads[servo_motor_e].thread_Stack, THREAD_STACK_SIZE,
    5, 5, TX_NO_TIME_SLICE, TX_AUTO_START) != TX_SUCCESS)
        status = TX_THREAD_ERROR;
    if (status == TX_THREAD_ERROR)
    {
        sprintf(err_msg, "Failservmt\r\n");
        HAL_UART_Transmit(&huart1, (uint8_t*)err_msg, strlen(err_msg), HAL_MAX_DELAY);
    }

                                        //SPEED SENSOR THREAD
    if (tx_thread_create(&threads[speed_sensor_e].thread_ptr, "speedS_thread", speed_sensor, 0, threads[speed_sensor_e].thread_Stack, THREAD_STACK_SIZE,
    6, 6, TX_NO_TIME_SLICE, TX_AUTO_START) != TX_SUCCESS)
        status = TX_THREAD_ERROR;
    if (status == TX_THREAD_ERROR)
    {
        sprintf(err_msg, "FailSS\r\n");
        HAL_UART_Transmit(&huart1, (uint8_t*)err_msg, strlen(err_msg), HAL_MAX_DELAY);
    }
```

CAN Bus

We successfully configured the Raspberry Pi to recognize and initialize both can0 and can1 interfaces. The presence and basic functionality of both distinct CAN networks were verified using ip link show and candump, confirming a stable link-up for our dual CAN setup on the RPi.

```
can0: <NOARP,UP,LOWER_UP,ECHO> mtu 16 qdisc pfifo_fast state UP mode DEFAULT group default qlen 1000
 link/can
```

```
can1: <NOARP,UP,LOWER_UP,ECHO> mtu 16 qdisc pfifo_fast state UP mode DEFAULT group default qlen 1000
 link/can
```

Now all together

Thank you!